

ENGINEIQ

Engineering Intelligence Platform

Milestone 0 — Local Setup & Test Guide

Step-by-step: from zero to first live PR comment on Billable

Confidential · Version 1.0 · March 2026

Before You Start

This guide takes you from a clean machine to a working EngineIQ instance posting real AI-generated review comments on Billable pull requests. Every command is copy-paste ready. Do the steps in order — each one depends on the previous.

What M0 does

When a pull request is opened, updated, or reopened on the Billable GitHub repo, the following happens automatically:

- GitHub fires a webhook to your running EngineIQ API
- The API validates the HMAC signature to confirm the request is genuine
- The PR diff is fetched from GitHub using the GitHub App installation token
- The diff is sent to Azure OpenAI with an architecture-aware prompt
- The AI response is posted back as a structured review comment on the PR

GOAL By the end of this guide, open any PR on the Billable repo and within 90 seconds you will see an EngineIQ review comment posted automatically. That is Milestone 0 complete.

What you need before starting

Requirement	Notes	Have it?
.NET 8 SDK	dotnet.microsoft.com/download — version 8.x	☐
GitHub account	With admin access to the Billable org/repo	☐
Azure subscription	Azure OpenAI resource deployed in SA North or nearest available	☐
ngrok (free tier)	ngrok.com — exposes localhost to GitHub webhooks	☐
EngineIQ solution	The scaffold from the EngineIQ_Scaffold.zip, cloned/unzipped locally	☐

Step 1 — Register the GitHub App

The GitHub App is the identity EngineIQ uses to read PRs and post comments. You register it once on the Billable GitHub organisation. This takes about 15 minutes.

1.1

Open GitHub App creation

Do this while logged into the GitHub account that owns the Billable org

Navigate to:

URL
`https://github.com/organizations/YOUR-ORG/settings/apps/new`

NOTE

Replace YOUR-ORG with your actual Billable GitHub organisation name. If Billable is a personal repo rather than an org, use: `github.com/settings/apps/new`

1.2

Fill in the App registration form

Fill in each field as follows. Leave anything not listed here at its default value.

Field	Value to enter
GitHub App name	EngineIQ-Dev
Homepage URL	<code>http://localhost:5000</code>
Webhook URL	<code>https://PLACEHOLDER.ngrok-free.app/webhooks/github</code>
Webhook secret	Generate a random string — e.g. <code>openssl rand -hex 32</code> . Save it, you will need it shortly.

WEBHOOK URL

You do not have your ngrok URL yet. Enter any placeholder for now (e.g. `https://placeholder.ngrok.io/webhooks/github`). You will update this in Step 4 once ngrok is running.

1.3

Set Repository permissions

Scroll to the Permissions section. Set these two permissions and leave everything else as No access:

Permission	Level	Why
Repository: Pull requests	Read & Write	Post review comments
Repository: Contents	Read	Fetch PR diff content

1.4 Subscribe to events

Scroll to Subscribe to events. Tick only this one:

EVENT Pull request — this fires on open, synchronize (new commit pushed), and reopen.

1.5 Set install scope and create

Under "Where can this GitHub App be installed?" select Only on this account.

Click Create GitHub App.

What you should see

- ✓ You are redirected to the App settings page
- ✓ You can see the App ID (a number like 123456) at the top — copy this now

1.6 Generate and download the Private Key

On the App settings page, scroll to the bottom to the Private keys section.

- Click Generate a private key
- A .pem file downloads automatically — save it somewhere safe, e.g. ~/keys/engiq-dev.pem
- Open the .pem file in any text editor — it starts with -----BEGIN RSA PRIVATE KEY----- and ends with -----END RSA PRIVATE KEY-----
- You will need the entire contents of this file as a single secret value

IMPORTANT The private key cannot be retrieved again after this point. If you lose it, you must generate a new one. Keep it out of source control — never commit .pem files.

1.7 Install the App on the Billable repository

On the App settings page, click Install App in the left sidebar.

- Click Install next to your organisation
- Select Only select repositories
- Choose the Billable repository from the dropdown
- Click Install

What you should see

- ✓ App is now listed as installed on the Billable repository
- ✓ You have: App ID, Private Key (.pem), Webhook Secret — all three needed for Step 3

Step 2 — Get Your Azure OpenAI Details

EngineIQ sends the PR diff to Azure OpenAI and gets back a review. You need an Azure OpenAI resource with a GPT-4o deployment. If you already have one, go straight to Step 2.3.

2.1

Create Azure OpenAI resource

Skip this if you already have an Azure OpenAI resource

In the Azure Portal (portal.azure.com):

1. Search for Azure OpenAI in the top search bar
2. Click Create
3. Select your subscription and resource group
4. Region: choose South Africa North if available — otherwise East US or West Europe
5. Name: engiq-dev-openai (or any name)
6. Pricing tier: Standard S0
7. Click Review + Create, then Create

REGION NOTE

Azure OpenAI is not always available in South Africa North. For local development, any region works. Production deployment should be SA North for POPIA compliance.

2.2

Deploy the GPT-4o model

Once the resource is created, open it and navigate to Model deployments.

8. Click Manage Deployments — this opens Azure AI Foundry
9. Click + New deployment
10. Select model: gpt-4o
11. Deployment name: gpt-4o (use this exact name — it matches your config)
12. Click Deploy

What you should see

- ✓ Deployment status shows Succeeded
- ✓ Deployment name is exactly gpt-4o

2.3

Copy your endpoint and API key

Back in the Azure Portal on your Azure OpenAI resource:

- Click Keys and Endpoint in the left sidebar
- Copy the Endpoint — it looks like: `https://engiq-dev-openai.openai.azure.com/`
- Copy Key 1 — a long alphanumeric string

What you should see

- ✓ You have: Endpoint URL (ending in `.openai.azure.com/`)
- ✓ You have: API Key (KEY 1)
- ✓ You have: Deployment name (`gpt-4o`)

Step 3 — Configure Local Secrets

.NET User Secrets stores sensitive config outside your source tree so it never accidentally gets committed to GitHub. All six secrets must be set before the API will start.

3.1 Open a terminal in the API project directory

```
Terminal
# Navigate to the API project
cd path/to/engiq/src/EngineIQ.API
```

3.2 Set all six secrets

Run each of these commands, replacing the placeholder values with your actual credentials:

```
Terminal - run from src/EngineIQ.API
# 1. GitHub App ID (the number from Step 1.5)
dotnet user-secrets set "GitHub:AppId" "123456"

# 2. Private Key PEM - paste the ENTIRE file content as one value
# The value must include the header and footer lines
dotnet user-secrets set "GitHub:PrivateKeyPem" "-----BEGIN RSA PRIVATE
KEY-----
MIIEowIBAAKCAQEA...(your key content)...
-----END RSA PRIVATE KEY-----"

# 3. Webhook secret you created in Step 1.2
dotnet user-secrets set "GitHub:WebhookSecret" "your-webhook-secret-here"

# 4. Azure OpenAI endpoint from Step 2.3
dotnet user-secrets set "AzureOpenAI:Endpoint" "https://your-
resource.openai.azure.com/"

# 5. Azure OpenAI API key from Step 2.3
dotnet user-secrets set "AzureOpenAI:ApiKey" "your-api-key-here"

# 6. Deployment name - must match exactly what you created in Azure
dotnet user-secrets set "AzureOpenAI:DeploymentName" "gpt-4o"
```

PEM KEY TIP

The PEM key must be on multiple lines inside the secret value. On Windows PowerShell use single quotes around multiline values. On Mac/Linux wrap in '\$'...' or use a heredoc. The easiest approach: write the value to a temp file and use: `dotnet user-secrets set "GitHub:PrivateKeyPem" "$(cat ~/keys/engiq-dev.pem)"`

3.3 Verify all secrets are set

```
Terminal  
dotnet user-secrets list
```

What you should see

- ✓ Output shows all 6 keys: GitHub:AppId, GitHub:PrivateKeyPem, GitHub:WebhookSecret, AzureOpenAI:Endpoint, AzureOpenAI:ApiKey, AzureOpenAI:DeploymentName
- ✓ Values are shown truncated — confirm each key name is correct

If you see: Error: No secrets configured

Fix: You are not in the src/EngineIQ.API directory. cd to that folder and run the commands again.

Step 4 — Build and Run the API

4.1 Restore and build the solution

From the root of the EngineIQ solution (where the .sln file is):

```
Terminal - run from solution root
# Restore all NuGet packages
dotnet restore

# Build the full solution
dotnet build
```

What you should see

- ✓ Build succeeded
- ✓ 0 Error(s)
- ✓ Warnings about stub projects are fine — these are M1+ placeholders

If you see: error CS0246: The type or namespace name could not be found

Fix: Run dotnet restore first. If it persists, check that all project references in the .sln file point to existing folders.

If you see: error NETSDK1045: The current .NET SDK does not support targeting .NET 8

Fix: Install .NET 8 SDK from dotnet.microsoft.com/download and restart your terminal.

4.2 Run the API

```
Terminal
# From solution root
dotnet run --project src/EngineIQ.API
```

What you should see

- ✓ Output shows: Now listening on: http://localhost:5000
- ✓ Output shows: Now listening on: https://localhost:5001
- ✓ No startup errors — if secrets are missing they will appear here

If you see: `System.InvalidOperationException: GitHub:AppId not configured`
Fix: One or more secrets are missing. Go back to Step 3 and run `dotnet user-secrets list` to see which ones are absent.

If you see: Port 5000 is already in use
Fix: Another process is on that port. Either kill it (`lsof -i :5000` on Mac/Linux, `netstat -ano` on Windows) or change the port in `Properties/launchSettings.json`.

4.3 Verify the API is healthy

Open a second terminal and run:

```
Terminal (second window)
curl http://localhost:5000/health
```

Or open `http://localhost:5000/health` in your browser.

What you should see

- ✓ Response: `{"status":"Healthy"}` or 200 OK
- ✓ API is reachable on port 5000

Step 5 — Expose Localhost with ngrok

GitHub needs a public HTTPS URL to send webhook events to. ngrok creates a secure tunnel from a public URL to your localhost:5000. You need this running whenever you are testing locally.

5.1 Install ngrok

```
Terminal
# Mac (Homebrew)
brew install ngrok/ngrok/ngrok

# Windows (Chocolatey)
choco install ngrok

# Or download directly from ngrok.com/download
```

Sign up for a free account at ngrok.com. Then authenticate:

```
Terminal
ngrok config add-authtoken YOUR_NGROK_TOKEN
# Your token is at: dashboard.ngrok.com/get-started/your-authtoken
```

5.2 Start the tunnel

In a new terminal window (keep the API running in the other one):

```
Terminal (third window)
ngrok http 5000
```

What you should see

- ✓ ngrok displays a Forwarding line like: `https://abc123.ngrok-free.app -> http://localhost:5000`
- ✓ The https URL is your public webhook URL — copy it

FREE TIER NOTE

ngrok free tier gives you a random URL that changes each time you restart ngrok. Every time you restart ngrok you must update the webhook URL in the GitHub App settings (Step 5.3). A paid ngrok account gives you a static domain.

5.3 Update the GitHub App webhook URL

Now that you have a real ngrok URL, update the placeholder you set in Step 1.2:

13. Go to github.com/settings/apps (or your org settings > Developer settings > GitHub Apps)
14. Click Edit on your EngineIQ-Dev app
15. In the Webhook URL field, paste: `https://YOUR-NGROK-URL.ngrok-free.app/webhooks/github`
16. Click Save changes

What you should see

- ✓ Webhook URL now shows your ngrok URL ending in `/webhooks/github`
- ✓ Webhook is Active (toggle should be green)

Step 6 — Fire the First PR and Verify

Everything is wired up. Now you trigger the real test: open a PR on the Billable repo and watch EngineIQ post a review comment.

6.1 Confirm all three processes are running

Before opening a PR, verify you have these three things active simultaneously:

#	What	Expected terminal output
1	<code>dotnet run --project src/EngineIQ.API</code>	Now listening on: http://localhost:5000
2	<code>ngrok http 5000</code>	Forwarding https://abc.ngrok-free.app -> localhost:5000
3	GitHub App webhook active	ngrok URL set in GitHub App settings

6.2 Open a test PR on the Billable repo

Create a simple branch, make any small code change, push it, and open a pull request:

```
Terminal - in your Billable repo
# In your Billable repo
git checkout -b test/engiq-m0-test

# Make any change - add a comment to any file
echo "// EngineIQ M0 test" >> src/SomeFile.cs

git add .
git commit -m "test: EngineIQ M0 trigger"
git push origin test/engiq-m0-test
```

Then go to GitHub and open a pull request from this branch to main.

6.3 Watch the API logs

Switch to the terminal running EngineIQ.API. Within a few seconds of opening the PR you should see:

```
API terminal output
```

```
info: EngineIQ.API.Controllers.WebhookController[0]
      Received GitHub webhook: pull_request / opened
info: EngineIQ.API.Controllers.WebhookController[0]
      HMAC signature valid
info: EngineIQ.GitHub.GitHubAppClient[0]
      Fetching diff: billable-org/billable-repo #42
info: EngineIQ.AIEngine.ReviewService[0]
      Sending diff to Azure OpenAI (gpt-4o)...
info: EngineIQ.AIEngine.ReviewService[0]
      Review received from OpenAI (1243 tokens)
info: EngineIQ.GitHub.GitHubAppClient[0]
      Comment posted to PR #42
```

6.4 Check the PR on GitHub

Go to the PR you just opened on GitHub. Scroll to the bottom of the conversation.

What you should see

- ✓ An EngineIQ review comment appears — posted by your GitHub App
- ✓ The comment contains an AI-generated review of the diff
- ✓ Comment shows the EngineIQ header and footer branding
- ✓ Total time from PR open to comment posted: under 90 seconds

M0 COMPLETE

If you see the review comment on the PR — Milestone 0 is done. The loop is real. EngineIQ is receiving webhooks, fetching diffs, calling Azure OpenAI, and posting structured review comments on Billable pull requests.

Troubleshooting

Work through these in order. Most issues fall into one of four categories: secrets, networking, GitHub App config, or Azure OpenAI.

No log output when PR is opened

The webhook is not reaching your API. Check each of these:

- Is ngrok running? Check the ngrok terminal — it should show Forwarding.
- Is the GitHub App webhook URL set to the current ngrok URL ending in /webhooks/github?
- Is the webhook marked Active in GitHub App settings (not disabled)?
- Check GitHub webhook delivery logs: Settings > Developer settings > GitHub Apps > Edit > Advanced > Recent Deliveries. Red = failed delivery. Click on a failed delivery to see the error response.

400 Bad Request in webhook delivery logs

The HMAC signature check is failing.

- The GitHub:WebhookSecret in your user-secrets must exactly match the Webhook Secret you entered in the GitHub App settings.
- Even a single space difference will cause HMAC mismatch.
- Re-run: `dotnet user-secrets set "GitHub:WebhookSecret" "your-exact-secret"`

If you see: 401 Unauthorized

Fix: HMAC signature is wrong. Regenerate the webhook secret in GitHub App settings and re-set it in user-secrets — use the same value in both places.

500 Internal Server Error — JWT / PEM issues

- The GitHub App private key must include the full PEM headers (-----BEGIN RSA PRIVATE KEY----- and -----END RSA PRIVATE KEY-----).
- Line breaks in the PEM must be preserved. If you set the secret on one line, the key will be malformed.
- Easiest fix on Mac/Linux:

```
Terminal — from src/EngineIQ.API
```



```
dotnet user-secrets set "GitHub:PrivateKeyPem" "$(cat ~/keys/engiq-dev.pem) "
```

- On Windows PowerShell:

```
PowerShell - from src/EngineIQ.API
$pem = Get-Content ~/keys/engiq-dev.pem -Raw
dotnet user-secrets set "GitHub:PrivateKeyPem" $pem
```

Azure OpenAI errors

If you see: `AuthenticationFailedException`

Fix: Your `AzureOpenAI:ApiKey` is wrong. Copy it again from Azure Portal > Keys and Endpoint > KEY 1.

If you see: `DeploymentNotFoundException` or 404

Fix: The deployment name in `AzureOpenAI:DeploymentName` does not match the deployment name in Azure AI Foundry. Check for typos — it must be exactly `gpt-4o`.

If you see: `RequestFailedException: Rate limit exceeded`

Fix: You have hit the Azure OpenAI rate limit. Wait 60 seconds and try again. For development, the default token limits are low. Increase quota in Azure AI Foundry if needed.

Review comment is empty or garbled

- Check that the diff was fetched successfully — look for "Fetching diff" in the API logs.
- If the diff is very large (> 15,000 tokens) the prompt may be truncated. The `ReviewPromptBuilder` limits diff length — this is expected behaviour.
- If the AI returns no findings, it will post a clean bill of health comment. This is correct — the test change (adding a code comment) may genuinely have no architectural issues.

Quick diagnostics checklist

#	Check	OK?
1	API is running and listening on localhost:5000	☐
2	ngrok is running and showing Forwarding line	☐

3	GitHub App webhook URL matches current ngrok URL (ends in /webhooks/github)	☐
4	GitHub App webhook is Active (not disabled)	☐
5	GitHub App is installed on the Billable repository	☐
6	All 6 user-secrets are set (dotnet user-secrets list shows 6 entries)	☐
7	GitHub:WebhookSecret matches exactly what is in GitHub App settings	☐
8	GitHub:PrivateKeyPem includes full PEM headers and line breaks	☐
9	AzureOpenAI:Endpoint ends with a trailing slash (/)	☐
10	AzureOpenAI:DeploymentName matches exactly the deployment in Azure AI Foundry	☐
11	PR was opened on the specific repository the GitHub App is installed on	☐
12	GitHub webhook delivery logs show 200 OK for recent deliveries	☐

What Comes Next — Milestone 1

With M0 working, the core loop is proven. Milestone 1 shifts from pure AI review to a hybrid model: deterministic rules run first, AI analysis runs second, and findings are merged, deduplicated, and severity-classified before posting.

M1 builds on M0 in these areas

Component	What gets built
EngineIQ.Domain	Finding entity, ArchitectureRule entity, FindingSeverity enum, ITenantRepository interface
EngineIQ.StandardsEngine	YAML config loader, rule executor — secret detection and async pattern checks live and tested
EngineIQ.ReviewEngine	Orchestrator that runs Standards Engine first, then AI, then merges and deduplicates findings
EngineIQ.FeedbackGenerator	Structured PR comment formatter: Critical / High / Medium / Suggestion sections with rule IDs and AI source labels
EngineIQ.Infrastructure	PostgreSQL repositories for Tenant and Finding persistence. Finding metadata stored — never source code.
EngineIQ.Worker	Service Bus consumer — dequeues PR jobs and drives the review pipeline asynchronously

M1 deliverable

DELIVERABLE	When a PR violates ARCH-001 (domain depending on infrastructure) or contains a hardcoded secret (SEC-001), the review comment shows it as a Critical finding with the rule ID, file path, line number, and a specific fix suggestion — separate from and in addition to the AI analysis.
--------------------	--

Keep these metrics from day one

Start measuring from the first M0 PR. These numbers become your sales asset:

- Time from webhook received to comment posted (target: under 90 seconds)
- Number of findings per PR — Critical, High, Medium, Suggestion
- Whether each finding was actioned or overridden by the developer
- PR cycle time before vs after EngineIQ was enabled

After 30 days on Billable you will have real data to put in front of prospective clients.